

Student

I know how to write `try/except` — but my senior engineer looked at my code and said it was 'amateur exception handling.' What does professional exception handling actually look like?

ChatGPT • AI Assistant

CONCEPT CARD

That's a common gap between beginners and professionals. Most beginners write `try/except` reactively — just to stop crashes. Professionals write exception handling as part of the design:

- They know **exactly** which exceptions to catch — and catch nothing else
- They use `else` and `finally` clauses deliberately
- They **chain exceptions** to preserve debugging context
- They design **custom exception hierarchies** that carry structured information
- They know when to catch and when to **let exceptions propagate**

The golden rule: an unhandled exception that crashes with a full traceback is almost always better than silently swallowing an error and producing wrong results. Exception handling is about *communication* — between code layers, between developers, and between your system and its operators.

Student

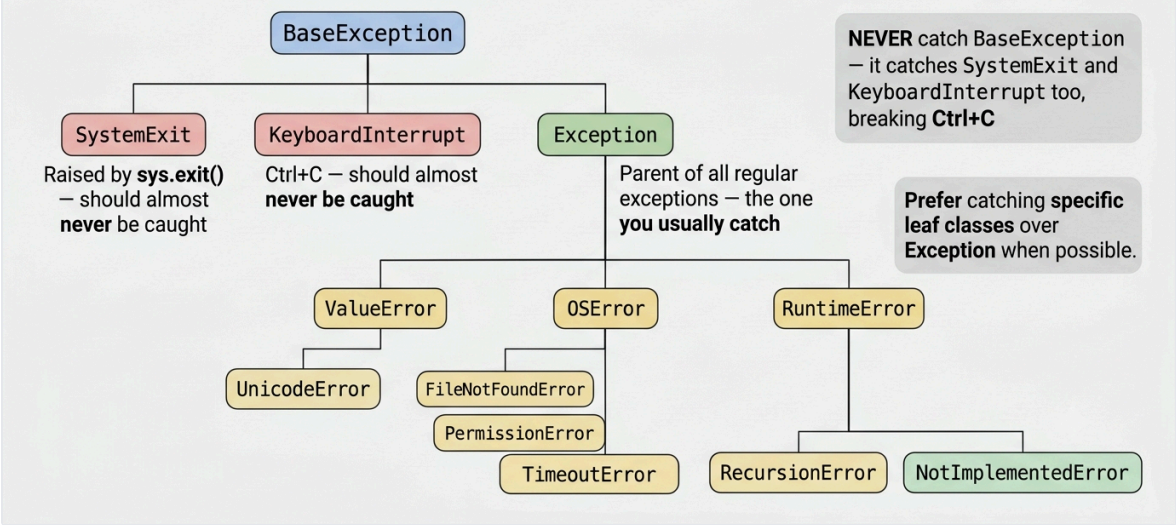
Why does it matter so much which exceptions I catch?

ChatGPT • AI Assistant

🔗 ANALOGY

Imagine you're a doctor. A patient comes in with a broken arm. You wouldn't say 'I'll treat ANY medical condition that walks in the door.' You'd diagnose specifically. Broad exception catching (`except Exception`) is like treating every possible illness with the same pill — you might accidentally suppress something serious and the patient walks out thinking they're fine while something critical is being ignored.

Understanding why catching Exception vs BaseException matters, and how to pick the right level



Python Exception Hierarchy — Built-in Exception Tree

try/except/else/finally — When Each Clause Runs

Clause	Runs When	Common Use	Runs if exception?
try	Always — first	Code that might raise	Yes (then stops at exception)
except	Only if matching exception raised	Handle / log / recover	Yes
else	Only if NO exception was raised	Post-success logic (DB commit, return result)	No
finally	Always — last, no matter what	Cleanup (close file, release lock)	Yes
raise (bare)	Inside except block	Re-raise the same exception after logging	Yes

THE FULL TRY/EXCEPT/ELSE/FINALLY BLOCK — WHEN EACH CLAUSE RUNS

ChatGPT • AI Assistant

A flowchart showing four separate execution scenarios side by side. Scenario 1 (green, 'No exception'): try runs fully → else runs → finally runs → continues. Scenario 2 (orange, 'Exception caught'): try raises → matching except runs → (else skipped) → finally runs → continues. Scenario 3 (red, 'Exception not caught'): try raises → no matching except → (else skipped) → finally runs → exception propagates. Scenario 4 (purple, 'return in try'): try hits return → (else skipped) → finally runs → return value delivered. Each scenario uses colored arrows showing which blocks execute (solid line) and which are skipped (dashed line). A highlighted box to the right: 'else = success-only code. finally = always code.' Learning objective: Develop an exact mental model of which clauses execute in every possible scenario.

try/except/else/finally — Execution Flow for All Cases

Student

I know `try` and `except` but I've never used `else` in a try block. What does it do?

ChatGPT • AI Assistant

CONCEPT CARD

The `else` clause in a `try` block is Python's most overlooked feature. It runs **only if the `try` block completed without raising any exception**. It's the perfect place for code that should only run on success — and crucially, any exception raised in `else` is NOT caught by the preceding `except` clauses. This is intentional: it separates 'this might fail' (try) from 'this should only run if it succeeded' (else).

EXAMPLE CARD

Example 1 — Basic: Seeing all four clauses in action

python — Code Example

```
def read_config(filepath):
    try:
        f = open(filepath)           # Might raise FileNotFoundError
        data = f.read()              # Might raise IOError
        f.close()
    except FileNotFoundError:
        print('Config file not found — using defaults')
        return {}                   # Return early with defaults
    except IOError as e:
        print(f'Error reading config: {e}')
        return {}
    else:
        # ONLY runs if open() and read() both succeeded — no
        # exceptions
```

```

print('Config loaded successfully')
import json
return json.loads(data)      # Parse the content
finally:
    # ALWAYS runs – success, exception, or even return!
    print('read_config() finished')

# Case 1: File exists and is valid
result = read_config('config.json')
# Output:
# Config loaded successfully
# read_config() finished

# Case 2: File not found
result = read_config('missing.json')
# Output:
# Config file not found – using defaults
# read_config() finished

```

Student

Why not just put the `json.loads(data)` inside the `try` block itself?

ChatGPT • AI Assistant

You *could* — but here's the subtle benefit of `else` : if `json.loads(data)` raises a `json.JSONDecodeError` , it will NOT be caught by the `except FileNotFoundError` or `except IOError` above it. Without `else` , a JSON parsing error could be accidentally swallowed by a broad `except` clause. Using `else` signals clearly: this code runs after success, and its exceptions are intentionally separate.

EXAMPLE CARD

Example 2 — Intermediate: The finally clause and return interaction

python — Code Example

```

def fetch_data(url):
    connection = None
    try:
        connection = open_connection(url)
        return connection.get_data()      # Return is inside try!
    except ConnectionError as e:
        print(f'Failed to connect: {e}')

```

```

    return None
finally:
    # This runs EVEN WHEN return is called in try or except!
    if connection:
        connection.close()
    print('Connection cleanup done')

# The return value from try is 'held' while finally runs,
# then the return actually happens. If finally has its own
# 'return', it OVERRIDES the try's return – a common gotcha!

def dangerous_finally():
    try:
        return 'try value'
    finally:
        return 'finally value' # WARNING: overrides try's return!

print(dangerous_finally()) # 'finally value' – surprising!

```

Student

Wait — `finally` can override a `return` ? That seems dangerous!

ChatGPT • AI Assistant

It can, and it almost always indicates a bug. This is why the rule is: **never use `return` inside a `finally` block**. Use `finally` only for cleanup — closing files, releasing locks, logging. If you return from `finally`, you also accidentally suppress any exception that was propagating, because a `return` statement inside `finally` stops the exception.

EXAMPLE CARD

Example 3 — Practical: A well-structured file parser using all four clauses

python – Code Example

```

import json

def parse_user_data(filepath):
    """
    Demonstrates professional use of all four clauses.
    try – the risky operations
    except – specific, targeted exception handling

```

```

else    - success-only logic, isolated from the try block
finally - guaranteed cleanup
"""

file_handle = None
try:
    file_handle = open(filepath, 'r', encoding='utf-8')
    raw_content = file_handle.read()
except FileNotFoundError:
    raise    # Re-raise - let the caller decide what to do
except PermissionError as e:
    raise RuntimeError(f'Cannot read {filepath}: permission
denied') from e
except UnicodeDecodeError as e:
    raise ValueError(f'File {filepath} is not valid UTF-8') from e
else:
    # Runs ONLY on successful file read
    # JSONDecodeError here is NOT caught by the excepts above
    parsed = json.loads(raw_content)
    # Validate required fields
    if 'users' not in parsed:
        raise KeyError('Missing required key: users')
    return parsed['users']
finally:
    # Always close the file - even if else raises!
    if file_handle:
        file_handle.close()

# Usage:
try:
    users = parse_user_data('users.json')
    print(f'Loaded {len(users)} users')
except FileNotFoundError:
    print('File missing - run setup first')
except ValueError as e:
    print(f'Data format error: {e}')

```

❗ COMMON PITFALL / MISTAKE

✗ Incorrect — catching too broadly, hiding bugs:

python - Code Example

```

def load_config(path):
    try:
        with open(path) as f:
            return json.loads(f.read())

```

```

except Exception:           # Catches EVERYTHING:
    return {}               # FileNotFoundError? Silenced.
                           # JSONDecodeError? Silenced.
                           # PermissionError? Silenced.
                           # Typo causing NameError? Also silenced!
# You will NEVER know why configs aren't loading in production.

```

✓ Correct — catch specifically, fail loudly on the unexpected:

python — Code Example

```

def load_config(path):
    try:
        with open(path) as f:
            content = f.read()
    except FileNotFoundError:
        return {} # Expected case: no config file yet
    # JSONDecodeError propagates – it's a programming/data error
    # that should be visible, not silently masked
    return json.loads(content)

```

INTERVIEW CARD

- **Frequency:** Very high — asked in nearly every Python mid-level interview
- **Question:** 'Explain the purpose of the `else` clause in a try/except block. Why would you use it instead of just putting the code at the end of the `try` block?'
- **Expected Answer:** `else` runs only when no exception was raised in `try`. Exceptions in `else` are NOT caught by the preceding `except` clauses. This cleanly separates 'code that might fail' (try) from 'code that should only run on success and whose failures are separate concerns' (else).
- **Follow-up:** 'What happens if `finally` contains a `return` statement when the `try` block also has a `return` ?'

EXERCISE

Refactor this code to use all four clauses properly — the current version has several issues:

python — Code Example

```

# BROKEN VERSION – find and fix all issues
def process_payment(amount, account_id):
    try:
        account = db.get_account(account_id)

```

```

    if account.balance < amount:
        raise ValueError('Insufficient funds')
    account.balance -= amount
    db.save(account)
    send_confirmation_email(account.email) # Should this be in
try?

    db.close() # Should this be in try?
    return True
except Exception as e: # Too broad!
    print(e)
    return False # Silently swallowed!

```

Hint: Which operations belong in **try** ? Which in **else** ? Which in **finally** ? Which exceptions are expected vs unexpected?

🔗 PRACTICE QUIZ

Concept Questions:

1. In which situation does the **else** clause of a try/except run?
2. What is the effect of placing a **return** statement inside a **finally** block?
3. Which clause is the right place for resource cleanup code, and why?

Output Prediction Questions:

1. What does this print?

python — Code Example

```

def f():
    try:
        x = int('not a number')
    except ValueError:
        print('caught ValueError')
        return 'from except'
    else:
        print('else ran')
    finally:
        print('finally ran')

print(f())

```

1. What does this print?

python — Code Example

```
def g():  
    try:  
        return 1  
    finally:  
        return 2  
  
print(g())
```

Debugging Question:

1. This function silently returns `None` when the database is down, causing confusing bugs downstream:

python – Code Example

```
def get_user(user_id):  
    try:  
        return db.fetch(user_id)  
    except Exception:  
        pass # What's the bug here?
```

What is the problem and what are two acceptable fixes?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. `else` runs only when the `try` block completes without raising ANY exception.
2. A `return` in `finally` overrides any `return` in `try` or `except`, and it also silently suppresses any exception that was propagating. Never use `return` in `finally`.
3. `finally` — because it runs unconditionally, whether the block succeeded, raised, or even called `return` or `break`.
4. Prints: `caught ValueError`, `finally ran`, then `from except`. The `else` clause does NOT run because an exception was raised.
5. Prints `2` — the `finally` return overrides the `try` return.
6. Problem: `except Exception: pass` silently swallows the database error. The caller receives `None` with no indication anything went wrong and may proceed with `None.name` etc. Fix 1: `except Exception: return None` with a comment

explaining this is intentional (rare). Fix 2 (better): remove the broad catch entirely and let the `DatabaseError` propagate to the caller who can decide what to do.

🔍 CHECKPOINT QUESTION

Look at this code and explain in one sentence why `else` is a better choice than putting `json.loads(content)` inside the `try` block:

python — Code Example

```
try:
    content = open(path).read()
except FileNotFoundError:
    return {}
else:
    return json.loads(content) # JSONDecodeError won't be caught
above
```

EXCEPTION CHAINING — RAISE FROM AND PRESERVING TRACEBACKS

ChatGPT • AI Assistant

Three vertical traceback panels side by side. Panel 1 (grey background, labeled 'No chain — raise X inside except'): shows traceback with 'ValueError: invalid literal' at top, then 'During handling of the above exception, another exception occurred:', then 'RuntimeError: something went wrong'. Text below: '.__context__ set implicitly'. Panel 2 (green background, labeled 'Explicit chain — raise X from Y'): shows traceback with 'ValueError: invalid literal' at top, then 'The above exception was the direct cause of the following exception:', then 'RuntimeError: something went wrong'. Text below: '.__cause__ set explicitly — intentional chain'. Panel 3 (blue background, labeled 'Suppress — raise X from None'): shows only 'RuntimeError: something went wrong'. Text below: 'No context shown — __cause__ = None, __suppress_context__ = True'. Bottom row shows when to use each: Panel 1 = incidental, Panel 2 = intentional translation, Panel 3 = library public API hiding internals. Learning objective: Read tracebacks and immediately identify chaining intent.

Exception Chaining — Traceback Comparison: No Chain vs from Y vs from None

Student

I've seen `raise ValueError('message') from original_error` in some code. What does `from` do here?

ChatGPT • AI Assistant

CONCEPT CARD

Exception chaining with `raise X from Y` does two things:

1. It raises a new exception `X`
2. It sets `X.__cause__ = Y` — explicitly linking the new exception to the original one

When Python prints the traceback, it shows BOTH exceptions: the original cause and the new one you raised. This is critical for debugging because it answers the question: 'Yes, a `ValueError` was raised — but WHY? What triggered it?' Without chaining, the original context is lost.

EXAMPLE CARD

Example 1 — Basic: The problem without chaining

python — Code Example

```
# WITHOUT chaining — original context lost
def load_user(user_id):
    try:
        data = database.fetch(f'SELECT * FROM users WHERE id={user_id}')
    except database.DatabaseError:
        raise ValueError(f'Invalid user ID: {user_id}')
        # Traceback shows ONLY ValueError.
        # The original DatabaseError — what SQL error actually
happened?
        # What was the connection state? All gone. Debugging
nightmare.

# Traceback (most recent call last):
#   File "app.py", line 10, in load_user
# ValueError: Invalid user ID: 42
# (You have NO idea what the database actually said)
```

EXAMPLE CARD

Example 2 — Intermediate: With exception chaining

python — Code Example

```
# WITH chaining — full debugging context preserved
def load_user(user_id):
    try:
        data = database.fetch(f'SELECT * FROM users WHERE id={user_id}')
    except database.DatabaseError as e:
        raise ValueError(f'Invalid user ID: {user_id}') from e
        # Now the traceback shows BOTH:
        # 1. The original DatabaseError (what the DB actually said)
```

2. Your ValueError (what it means at the application level)

```
# Traceback:
# database.DatabaseError: connection refused (host=db.prod:5432)
#
# The above exception was the direct cause of the following exception:
#
# Traceback (most recent call last):
#   File "app.py", line 10, in load_user
# ValueError: Invalid user ID: 42
#
# Now you know EXACTLY what happened: DB refused connection → caused
# ValueError
```

Student

When should I use `raise X from Y` vs just `raise X` ?

ChatGPT • AI Assistant

Here is the decision guide:

- Use `raise X from Y` when you're **translating** a lower-level exception into a higher-level one for your caller. The original exception caused the new one — preserve that chain.
- Use `raise X from None` when you **intentionally want to hide** the original exception (rare — e.g., hiding internal implementation details from library users).
- Use bare `raise` (no argument) to **re-raise** the current exception unchanged — useful for logging before propagating.
- Never create a new exception and raise it without `from` inside an `except` block — Python will implicitly chain it anyway (as `__context__`), but explicitly using `from` makes the intent clear.

EXAMPLE CARD

Example 3 — Practical: Exception translation in a data pipeline

python — Code Example

```
import csv

class DataPipelineError(Exception):
    """Base for all pipeline errors."""
    pass
```

```

class ParseError(DataPipelineError):
    """Raised when input data cannot be parsed."""
    def __init__(self, message, line_number=None, raw_value=None):
        super().__init__(message)
        self.line_number = line_number
        self.raw_value = raw_value

def parse_csv_record(row, line_number):
    """
    Translates low-level exceptions into domain-specific ones.
    The original exception is chained for full debugging context.
    """
    try:
        return {
            'id': int(row['id']),          # Might raise
ValueError
            'age': int(row['age']),        # Might raise
ValueError
            'score': float(row['score']),  # Might raise ValueError
            'name': row['name'].strip()    # Might raise KeyError
        }
    except ValueError as e:
        raise ParseError(
            f'Line {line_number}: invalid numeric value',
            line_number=line_number,
            raw_value=str(row)
        ) from e      # Chain: original ValueError is preserved
    except KeyError as e:
        raise ParseError(
            f'Line {line_number}: missing required field {e}',
            line_number=line_number,
            raw_value=str(row)
        ) from e      # Chain: original KeyError is preserved

# Usage:
try:
    record = parse_csv_record({'id': 'abc', 'age': '25', 'score':
'9.5', 'name': 'Alice'}, line_number=42)
except ParseError as e:
    print(f'Parse failed at line {e.line_number}: {e}')
    print(f'Raw value was: {e.raw_value}')
    print(f'Original cause: {e.__cause__}')

# Output:
# Parse failed at line 42: Line 42: invalid numeric value

```

```
# Raw value was: {'id': 'abc', 'age': '25', ... }
# Original cause: invalid literal for int() with base 10: 'abc'
```

EXAMPLE CARD**Example 4 — Advanced: Implicit vs explicit chaining**

python — Code Example

```
# Python IMPLICITLY chains when you raise inside an except block
try:
    int('bad')
except ValueError:
    raise RuntimeError('something went wrong')
# Traceback shows:
# ValueError: invalid literal...
# During handling of the above exception, another exception occurred:
# RuntimeError: something went wrong
# NOTE: 'During handling...' is IMPLICIT chaining (__context__)

# EXPLICIT chaining with 'from' shows:
# ValueError: invalid literal...
# The above exception was the direct cause of the following exception:
# RuntimeError: something went wrong
# NOTE: 'direct cause' is EXPLICIT chaining (__cause__)

# The phrase 'direct cause' vs 'during handling' tells readers
# whether the chaining was intentional (from) or incidental.

# Suppress chaining entirely with 'from None':
try:
    import internal_module
except ImportError as e:
    raise PublicAPIError('Feature not available') from None
# Only PublicAPIError shown — ImportError hidden from library users
```

Student

Is this asked in interviews?

ChatGPT • AI Assistant

 INTERVIEW CARD

- **Frequency:** Medium — separates good Python developers from great ones
- **Question:** 'What is the difference between `raise X` , `raise X from Y` , and `raise X from None` ?'
- **Expected Answer:** `raise X` raises a new exception; if inside an `except` block, Python implicitly sets `X.__context__` to the current exception (implicit chaining). `raise X from Y` explicitly sets `X.__cause__ = Y` , showing 'direct cause' in the traceback — use when exception X was directly caused by Y. `raise X from None` suppresses chaining entirely — use to hide implementation details in library code.
- **Follow-up:** 'Where are `__cause__` and `__context__` stored on the exception object?'

❗ COMMON PITFALL / MISTAKE

✗ Incorrect — losing context by not chaining:

python – Code Example

```
except requests.ConnectionError:
    raise ServiceUnavailableError('Payment service down')
# Developer sees ServiceUnavailableError in production.
# What was the actual network error? Timeout? DNS failure? SSL error?
# All context lost. Debugging takes hours instead of minutes.
```

✓ Correct — chain to preserve context:

python – Code Example

```
except requests.ConnectionError as e:
    raise ServiceUnavailableError('Payment service down') from e
# Full traceback: shows BOTH the requests.ConnectionError details
# AND the ServiceUnavailableError. Debug in minutes.
```

🔥 EXERCISE

You are building an API client. Write a function `get_user_profile(user_id)` that:

1. Makes an HTTP request (simulate with `requests.get`)
2. Catches `requests.HTTPError` and translates it to a domain-specific `UserNotFoundError` if the status is 404, or a `ServiceError` for any other HTTP error
3. Chains the original exception in both cases
4. Lets `requests.ConnectionError` propagate unchanged (no catch)

python — Code Example

```

class UserNotFoundError(Exception): pass
class ServiceError(Exception): pass

def get_user_profile(user_id):
    # TODO: implement with proper exception chaining
    pass

```

🔗 PRACTICE QUIZ

Concept Questions:

1. What attribute does `raise X from Y` set on exception `X` ?
2. What phrase does Python use in the traceback for `raise X from Y` vs `raise X` (inside except)?
3. When is `raise X from None` appropriate?

Output Prediction Questions:

1. After `raise ValueError('bad') from KeyError('missing')` , what is the value of `err.__cause__` ?
2. What does bare `raise` do when called inside an `except` block?

Debugging Question:

1. A developer catches a database error and raises a custom one, but in production the traceback shows only the custom exception and the team can never diagnose the real DB issue. What one-word fix is needed?

python — Code Example

```

except db.OperationalError:
    raise DataStorageError('Database unavailable') # Fix here

```

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. `raise X from Y` sets `X.__cause__ = Y` .
2. `raise X from Y` shows 'The above exception was the **direct cause** of the following exception'. `raise X` inside except shows 'During handling of the above

exception, **another exception occurred**'.

3. `raise X from None` is appropriate when you intentionally want to hide internal implementation details — e.g., a library catching its own internal errors and raising a clean public API exception without exposing internal module structure.
4. `err.__cause__` is the `KeyError('missing')` instance.
5. Bare `raise` re-raises the currently active exception — same type, same message, same traceback — unchanged.
6. Add `from e` (or `from the caught exception`): `raise DataStorageError('Database unavailable') from e`.

🔍 CHECKPOINT QUESTION

You are building a payment library. Your internal code uses `stripe.StripeError`. Your public API raises `PaymentError`. Should you use `raise PaymentError( ... ) from stripe_error` or `raise PaymentError( ... ) from None`? What is the tradeoff?

DESIGNING CUSTOM EXCEPTION HIERARCHIES

ChatGPT • AI Assistant

A tree diagram. Root node (blue, wide): 'Exception (Python built-in)'. One child node (green, wide): 'DataPipelineError — attrs: message, context dict'. Three child nodes beneath it: Node 1 (orange): 'NetworkError — attrs: url, status_code'. Node 2 (red): 'SchemaValidationError — attrs: field_name, expected_type, actual_value'. Node 3 (purple): 'DataQualityError — attrs: rule_name, offending_records (list)'. To the right of the tree, a caller code box shows: 'except NetworkError → retry or alert ops', 'except SchemaValidationError → alert data team', 'except DataQualityError → quarantine records', 'except DataPipelineError → safety net for future subclasses'. Arrows from each caller box point to the corresponding tree node. Bottom note: 'Callers choose the level of specificity they need.' Learning objective: See how a hierarchy enables both specific and broad exception handling from the same codebase.

Custom Exception Hierarchy — DataPipelineError Tree with Attributes

Student

When should I create my own exception classes instead of using Python's built-in ones?

ChatGPT • AI Assistant

CONCEPT CARD

Create custom exceptions when:

- You need callers to be able to **catch your errors specifically** without catching every `ValueError` in existence
- Your errors carry **structured context** — not just a message string, but attributes like `line_number`, `field_name`, `status_code`
- You're building a **library or module** and want to expose a clean error surface
- You want to organize errors into a **hierarchy** so callers can catch at the right level of specificity

The standard pattern is a **base exception class** for your module, with specific subclasses for each error category. Callers who want to handle everything use the base class; those who need to distinguish use the subclasses.

EXAMPLE CARD

Example 1 — Basic: Simple custom exception

python — Code Example

```
# Minimal custom exception — just a name, no extra behavior
class AppError(Exception):
    """Base class for all application errors."""
    pass

class ConfigError(AppError):
    """Raised when configuration is invalid or missing."""
    pass

# Usage:
try:
    raise ConfigError('Missing DATABASE_URL environment variable')
except AppError as e:
    print(f'App error: {e}')    # Catches both AppError AND
                               # ConfigError
except Exception as e:
    print(f'Unexpected: {e}')  # Only for truly unexpected errors
```

EXAMPLE CARD

Example 2 — Intermediate: Custom exceptions with structured attributes

python — Code Example

```
class DataPipelineError(Exception):
    """
    Base class for all data pipeline errors.
    All subclasses carry a 'context' dict for structured debugging.
```

```

"""
def __init__(self, message, context=None):
    super().__init__(message)
    self.context = context or {}

def __str__(self):
    base = super().__str__()
    if self.context:
        ctx_str = ', '.join(f'{k}={v}' for k, v in
self.context.items())
        return f'{base} [{ctx_str}]'
    return base

class NetworkError(DataPipelineError):
    """
    Raised when a network or API call fails.
    Extra attributes: status_code, url
    """
    def __init__(self, message, url=None, status_code=None, **kwargs):
        super().__init__(message, context={
            'url': url,
            'status_code': status_code,
            **kwargs
        })
        self.url = url
        self.status_code = status_code

class SchemaValidationError(DataPipelineError):
    """
    Raised when incoming data fails schema validation.
    Extra attributes: field_name, expected_type, actual_value
    """
    def __init__(self, message, field_name=None, expected_type=None,
actual_value=None):
        super().__init__(message, context={
            'field': field_name,
            'expected': expected_type,
            'actual': repr(actual_value)
        })
        self.field_name = field_name
        self.expected_type = expected_type
        self.actual_value = actual_value

```

```
class DataQualityError(DataPipelineError):
    """
    Raised when data passes schema but fails quality checks.
    Extra attributes: rule_name, offending_records
    """
    def __init__(self, message, rule_name=None,
offending_records=None):
        super().__init__(message, context={
            'rule': rule_name,
            'offending_count': len(offending_records or [])
        })
        self.rule_name = rule_name
        self.offending_records = offending_records or []
```

EXAMPLE CARD**Example 3 — Practical: Using the hierarchy in a real pipeline**

python — Code Example

```
import requests

def fetch_raw_data(api_url):
    try:
        response = requests.get(api_url, timeout=5)
        response.raise_for_status()
        return response.json()
    except requests.Timeout:
        raise NetworkError(
            'API request timed out',
            url=api_url,
            status_code=None
        ) from None
    except requests.HTTPError as e:
        raise NetworkError(
            f'API returned error status',
            url=api_url,
            status_code=e.response.status_code
        ) from e

def validate_record(record, schema):
    for field, expected_type in schema.items():
        if field not in record:
            raise SchemaValidationError(
                f'Missing required field',
                field_name=field,
```

```

        expected_type=expected_type,
        actual_value=None
    )
    if not isinstance(record[field], expected_type):
        raise SchemaValidationError(
            f'Type mismatch for field {field}',
            field_name=field,
            expected_type=expected_type.__name__,
            actual_value=record[field]
        )

def check_quality(records):
    duplicates = [r for r in records if records.count(r) > 1]
    if duplicates:
        raise DataQualityError(
            'Duplicate records detected',
            rule_name='no_duplicates',
            offending_records=duplicates
        )

# Caller with different handling granularity:
def run_pipeline(api_url, schema):
    try:
        data = fetch_raw_data(api_url)
        for record in data:
            validate_record(record, schema)
        check_quality(data)
        return data
    except NetworkError as e:
        print(f'Network issue – retry later: {e}')
        # Log e.url, e.status_code for alerting
        raise
    except SchemaValidationError as e:
        print(f'Schema problem – fix the source: {e}')
        # Log e.field_name, e.expected_type for the data team
        raise
    except DataQualityError as e:
        print(f'Quality issue – {len(e.offending_records)} bad
records: {e}')
        raise
    except DataPipelineError as e:
        # Catch-all for any OTHER pipeline error subclass
        print(f'Unexpected pipeline error: {e}')
        raise

```

Why does the catch-all `except DataPipelineError` at the bottom not catch the earlier exceptions?

ChatGPT • AI Assistant

Because Python tries `except` clauses **in order** and stops at the first match. `NetworkError` is caught before `DataPipelineError` even though `NetworkError` IS a `DataPipelineError`. The catch-all at the bottom only fires for subclasses of `DataPipelineError` that don't match any earlier `except` clause — acting as a safety net for future subclasses you haven't written yet.

❗ COMMON PITFALL / MISTAKE

✗ Incorrect — raising base Exception directly:

python — Code Example

```
def fetch_data(url):
    try:
        ...
    except requests.ConnectionError:
        raise Exception('fetch failed') # Uncatchable specifically!
        # Callers must catch all Exception — too broad.
        # They can't distinguish 'fetch failed' from 'null pointer
        error'
```

✓ Correct — raise domain-specific exceptions:

python — Code Example

```
class FetchError(AppError): pass

def fetch_data(url):
    try:
        ...
    except requests.ConnectionError as e:
        raise FetchError(f'Failed to fetch {url}') from e
        # Callers can now: except FetchError (specific)
        # or: except AppError (broad for the whole module)
```

📄 INTERVIEW CARD

- **Frequency:** High for senior Python roles and API/library design interviews
- **Question:** 'Design a custom exception hierarchy for a data pipeline. What base class would you use, what subclasses, and what custom attributes would each carry?'
- **Expected Answer:** One base class inheriting from `Exception` (not `BaseException`). Subclasses for each logical error category: `NetworkError`, `SchemaValidationError`, `DataQualityError`. Each carries relevant context as attributes. Base class has a `context` dict for structured logging. All subclasses call `super().__init__(message)`.
- **Follow-up:** 'Why should your base class inherit from `Exception` and not directly from `BaseException`?'

🔥 EXERCISE

Design a custom exception hierarchy for an **e-commerce checkout system**. You need errors for:

1. Item out of stock (carries: `item_id`, `requested_qty`, `available_qty`)
2. Payment declined (carries: `amount`, `currency`, `decline_code`)
3. Shipping address invalid (carries: `address_field`, `reason`)
4. A base class `CheckoutError` that all three inherit from

Write the full class definitions. Then write a `process_checkout(cart, payment, address)` function that raises the appropriate exception type at each step.

🔍 PRACTICE QUIZ

Concept Questions:

1. Why should custom exception base classes inherit from `Exception` rather than `BaseException`?
2. Why is `super().__init__(message)` important in a custom exception's `__init__`?
3. If a caller does `except DataPipelineError`, will it also catch `NetworkError` (a subclass)? Why?

Output Prediction Questions:

1. Given the hierarchy above, which except clause fires here?

python — Code Example

```
try:
    raise SchemaValidationError('bad field', field_name='age')
```

```

except NetworkError:
    print('Network')
except DataPipelineError:
    print('Pipeline')
except Exception:
    print('Exception')

```

1. What does `str(SchemaValidationError('bad field', field_name='age', expected_type='int', actual_value='abc'))` return?

Debugging Question:

1. A developer's custom exception loses the message when printed:

python – Code Example

```

class MyError(Exception):
    def __init__(self, message, code):
        self.message = message    # Bug: forgot super().__init__!
        self.code = code

err = MyError('something broke', code=42)
print(err)                # Prints: (empty!)
print(str(err))           # Prints: (empty!)

```

Why is it empty and what is the fix?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. `BaseException` is the root of ALL Python exceptions including `SystemExit` and `KeyboardInterrupt`. If your base inherits from `BaseException`, callers could accidentally suppress Ctrl+C or `sys.exit()` by catching your base class.
2. `super().__init__(message)` sets the exception's `args` tuple, which is what `str(exception)` and the traceback display. Without it, `print(exc)` shows nothing.
3. Yes — `except DataPipelineError` catches `DataPipelineError` AND all its subclasses, including `NetworkError`. Python's `except` clause uses `issubclass` under the hood.
4. Prints `'Pipeline'` — `SchemaValidationError` is a `DataPipelineError`, so it matches that clause. `NetworkError` doesn't match.

5. Returns `'bad field [field=age, expected=int, actual='abc']'` — because of the custom `__str__` on the base class.
6. Bug: `super().__init__()` is never called, so `self.args` is empty. `Exception.__str__` returns `str(self.args[0])` if args has one element, or `str(self.args)`. With empty `args`, it returns `''`. Fix: add `super().__init__(message)` in `__init__`.

🔍 CHECKPOINT QUESTION

A teammate says: 'Just raise `ValueError` everywhere — why create custom exceptions?' Give two concrete reasons why a custom exception hierarchy is worth the extra code.

WHEN TO CATCH VS WHEN TO LET EXCEPTIONS PROPAGATE

ChatGPT • AI Assistant

A vertical three-layer architecture diagram. Top layer (blue box): 'API Handler / Route'. Middle layer (green box): 'Business Service'. Bottom layer (orange box): 'Repository / DB Client'. External system box at the very bottom (grey): 'Database'. Arrows between layers: Bottom arrow up: 'DatabaseConnectionError raised by DB driver'. At the Repository layer boundary, a transformation box: 'except DatabaseConnectionError → raise DataStorageError from e'. Arrow continues up through 'Business Service' layer with label 'DataStorageError propagates — Service does NOT catch'. At the API Handler boundary, another transformation box: 'except DataStorageError → return 503 JSON response'. To the right of the diagram, a color-coded rule box: Green check = 'Catch at boundaries (translate)'; Red cross = 'Do NOT catch in middle layers (no value added)'; Orange warning = 'Log unexpected exceptions at the top layer with exc_info=True'. Learning objective: Know exactly which layer catches what, and why the middle layer is a pass-through.

Exception Propagation in Layered Architecture — The Boundary Pattern

Student

This is the thing I struggle with the most. I never know whether to catch an exception or just let it crash. How do professionals decide?

ChatGPT • AI Assistant

CONCEPT CARD

This is the question that separates junior from senior developers. The core principle is: **catch an exception only if you can do something useful with it at that layer**. If you can't add value — can't recover, can't provide context, can't log meaningfully — then let it propagate to a layer that can.

The layers where catching makes sense:

1. **At the boundary** — where your code meets external systems (database, API, filesystem). Translate low-level exceptions into domain exceptions here.
2. **At the top** — in your main function, web framework request handler, or worker loop. Catch, log, and decide whether to continue or crash.
3. **For expected conditions** — `FileNotFoundError` when checking if a config file exists, `KeyError` when looking up an optional dict key.

Never catch exceptions in the middle layers just to silence them.

EXAMPLE CARD

Example 1 — Basic: The wrong place to catch — middle layer swallowing

python — Code Example

```
# WRONG: middle layer swallowing exceptions
class UserService:
    def get_user(self, user_id):
        try:
            return self.db.fetch_user(user_id) # DB might raise
        except Exception:
            return None # Silent failure – hides the problem
                        # completely

class OrderService:
    def create_order(self, user_id, items):
        user = self.user_service.get_user(user_id) # Returns None
        # silently
        order = Order(user=user, items=items)      # user is None!
        order.save() # NullPointerException-style crash HERE – far from
                    # the real cause
        # Stack trace points to Order() – not to the DB failure.
        # Hours of debugging to find the real cause.
```

EXAMPLE CARD

Example 2 — Intermediate: The right place to catch — at the boundary

python — Code Example

```
# CORRECT: catch at the boundary (DB layer), translate, propagate
class UserRepository:
    """Database boundary layer – translates DB exceptions to domain
    exceptions."""
```

```

def fetch_user(self, user_id):
    try:
        result = self.db.execute('SELECT * FROM users WHERE id=?',
            (user_id,))
        if not result:
            raise UserNotFoundError(f'No user with id={user_id}')
        return User.from_row(result[0])
    except DatabaseConnectionError as e:
        raise DataStorageError('Database unavailable') from e
    # ValueError, KeyError from User.from_row() propagate –
    # they're bugs

class UserService:
    """Business logic layer – does NOT catch exceptions, just uses the
    repo."""

    def get_user(self, user_id):
        return self.repo.fetch_user(user_id) # Let exceptions
        propagate!

class APIHandler:
    """Top-layer boundary – catches domain exceptions for HTTP
    responses."""

    def handle_get_user(self, user_id):
        try:
            user = self.user_service.get_user(user_id)
            return {'status': 200, 'user': user.to_dict()}
        except UserNotFoundError:
            return {'status': 404, 'error': 'User not found'}
        except DataStorageError:
            return {'status': 503, 'error': 'Service temporarily
            unavailable'}
        # Other exceptions propagate to the framework's global error
        handler

```

Student

So the pattern is: catch at the database layer, let it travel through the business logic layer, catch again at the API layer?

Exactly — this is the **boundary pattern**. Each architectural boundary is responsible for translating exceptions from the language of the layer below to the language of the layer above. DB layer speaks `DatabaseError` → translates to domain `DataStorageError`. API layer speaks HTTP status codes → translates domain exceptions to HTTP responses. The middle business logic layer is just a highway — it doesn't catch unless it has something specific to do.

EXAMPLE CARD

Example 3 — Practical: Global exception handler for a long-running worker

python — Code Example

```
import logging
import time

logger = logging.getLogger(__name__)

def process_job(job):
    """Individual job processing — raises specific domain
    exceptions."""
    # ... actual processing ...
    pass

def worker_loop(job_queue):
    """
    Top-level worker loop.
    Catches at the right level: per-job errors vs fatal errors.
    """
    while True:
        try:
            job = job_queue.get(timeout=1)
        except QueueEmpty:
            continue # Expected — no jobs right now, keep polling

        try:
            process_job(job) # Run the job
            job_queue.task_done(job) # Mark success
        except JobValidationError as e:
            # Expected failure — bad job data. Log, discard, continue.
            logger.warning(f'Invalid job {job.id}: {e}. Discarding.')
            job_queue.discard(job)
        except TransientError as e:
            # Expected failure — temporary issue. Log and retry.
            logger.warning(f'Transient error on job {job.id}: {e}.
```

```

    Requeueing.')
    job_queue.requeue(job, delay=30)
except Exception as e:
    # UNEXPECTED failure – log with full traceback, stop the
    worker.

    logger.critical(
        f'Unexpected error on job {job.id}. Worker stopping.',
        exc_info=True    # Includes full traceback in log
    )
    raise # Let the process supervisor restart the worker
# Note: KeyboardInterrupt and SystemExit are NOT caught here
# They propagate through and cleanly stop the worker

```

Student

I notice you use `exc_info=True` in the logger. What does that do?

ChatGPT • AI Assistant

`logging.critical(..., exc_info=True)` (or `logger.exception(...)` which always includes it) attaches the **full traceback** to the log message. Without it, you only get the exception message — no line numbers, no call stack. In production, without the traceback, debugging an unexpected error is nearly impossible. Always log with `exc_info=True` for unexpected exceptions.

EXAMPLE CARD

Example 4 — Real Business Example: Full exception handling in a Flask API endpoint

python – Code Example

```

from flask import Flask, jsonify
import logging

app = Flask(__name__)
logger = logging.getLogger(__name__)

@app.errorhandler(UserNotFoundError)
def handle_user_not_found(e):
    return jsonify({'error': str(e)}), 404

@app.errorhandler(DataStorageError)
def handle_storage_error(e):
    logger.error('Storage failure', exc_info=True)

```

```

    return jsonify({'error': 'Service temporarily unavailable'}), 503

@app.errorhandler(Exception)
def handle_unexpected(e):
    logger.critical('Unhandled exception', exc_info=True)
    return jsonify({'error': 'Internal server error'}), 500

@app.route('/users/<int:user_id>')
def get_user(user_id):
    # NO try/except here – exceptions propagate to the error handlers above
    user = user_service.get_user(user_id) # Let it raise!
    return jsonify(user.to_dict())

```

Student

The route function has NO exception handling at all! That seems wrong.

ChatGPT • AI Assistant

This is intentional and elegant. Flask (and most frameworks) have a global error handler registration system — `@app.errorhandler`. The route function focuses entirely on the happy path. Domain exceptions propagate to the framework's handler layer, which maps them to HTTP responses. This is the cleanest separation of concerns. Adding try/except inside every route function is repetitive and error-prone.

❗ COMMON PITFALL / MISTAKE

✗ Incorrect — catching in the middle layer with no recovery:

python – Code Example

```

class PaymentService:
    def charge(self, amount, card):
        try:
            return self.gateway.charge(amount, card)
        except PaymentGatewayError as e:
            logger.error(f'Payment failed: {e}') # Logs it ...
            return None                          # ... then
                                                # silently returns None!
                                                # Caller receives None and thinks charge succeeded
                                                # Order marked as paid. Customer never charged. Revenue lost.

```

✓ Correct — log AND re-raise:

python — Code Example

```
class PaymentService:
    def charge(self, amount, card):
        try:
            return self.gateway.charge(amount, card)
        except PaymentGatewayError as e:
            logger.error(f'Payment failed: {e}', exc_info=True)
            raise # Caller must handle this — it's not optional
```

🔗 INTERVIEW CARD

- **Frequency:** High for senior roles — tests architectural thinking
- **Question:** 'You have a three-layer architecture: repository, service, API handler. Where should database exceptions be caught, and why?'
- **Expected Answer:** At the repository (boundary) layer — translate DB-specific exceptions to domain exceptions. Service layer lets domain exceptions propagate — it has no context to add value. API handler catches domain exceptions and maps them to HTTP status codes. Each layer catches only what it can handle meaningfully.
- **Follow-up:** 'What is the problem with catching Exception in a middle service layer and returning None?'

🔗 AI DEEP DIVE PROMPT

Copy this into Claude or ChatGPT:

"I understand Python exception handling patterns. Now teach me production exception strategies: structured logging with exc_info, Sentry/error tracking integration, retry-with-backoff using custom exceptions, circuit breaker patterns using exception state, exception handling in async Python (asyncio), and exception groups (Python 3.11+). For each: (1) explain the production problem it solves, (2) show a minimal working example, (3) show a real-world scenario, (4) give an interview question. Challenge me with output prediction questions before revealing answers."

🔗 EXERCISE

Refactor this production order processing function — it has at least 5 exception handling problems:

python — Code Example

```
def process_order(order_id):
    try:
        order = db.get_order(order_id)
        inventory.reserve(order.items)
        payment.charge(order.total, order.card)
        shipping.schedule(order)
        db.mark_complete(order_id)
        email.send_confirmation(order.email)
        return True
    except Exception as e:
        print(f'Error: {e}')
        return False
```

Identify all 5 problems and write the correctly structured version.

🔍 PRACTICE QUIZ

Concept Questions:

1. What is the 'boundary pattern' for exception handling in layered architectures?
2. Why is logging with `exc_info=True` important for unexpected exceptions?
3. What is wrong with catching an exception, logging it, and then returning `None` ?

Output Prediction Questions:

1. In Flask's `@app.errorhandler` pattern, if the route function raises `UserNotFoundError` , what happens before the response is sent?
2. In the worker loop, why are `KeyboardInterrupt` and `SystemExit` intentionally not caught?

Debugging Question:

1. A developer's service catches errors and returns `False` on failure. The test suite passes. In production, orders are silently lost. Find the architectural bug:

python — Code Example

```
class OrderService:
    def place_order(self, cart):
        try:
            db.save(Order(cart))
            payment.charge(cart.total)
```

```

    return True
except Exception:
    return False # Where is the bug?

```

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. The boundary pattern: each architectural boundary (DB layer, external API layer, framework layer) is responsible for catching exceptions from the layer below, translating them to the vocabulary of the current layer, and propagating them upward. Middle layers do not catch — they let exceptions travel through.
2. `exc_info=True` attaches the full traceback (file names, line numbers, call stack) to the log entry. Without it, you only see the exception message, making production debugging nearly impossible.
3. The caller has no way to distinguish 'operation succeeded, result is None' from 'operation failed, None is the error indicator'. Silent `None` returns cause bugs far from the original failure point.
4. Flask catches the `UserNotFoundError`, finds the matching `@app.errorhandler(UserNotFoundError)`, calls that handler function, and sends the 404 JSON response it returns.
5. `KeyboardInterrupt` (Ctrl+C) and `SystemExit` (`sys.exit()`) are meant to terminate the process. Catching them in the worker loop would prevent clean shutdown, create zombie processes, and ignore operator intent.
6. If `payment.charge()` fails (card declined), the order was already saved to DB (line above). The function returns `False` but the incomplete order exists in the database — half-committed state. The caller sees `False` and tries again, potentially creating a duplicate order. Fix: use a database transaction so the entire operation is atomic, and propagate the payment exception rather than swallowing it.

SUMMARY CARD

- Catch only what you can handle meaningfully — let the rest propagate
- The boundary pattern: translate exceptions at layer boundaries, let them travel through middle layers
- Always log unexpected exceptions with `exc_info=True` for full tracebacks
- Never catch and return `None` silently — it creates debugging nightmares downstream

- Use `else` for success-only code — its exceptions are intentionally separate from the `except` handlers
- Use `raise X from Y` when translating exceptions — preserve the original cause for debugging
- Design custom exception hierarchies for library/module code — gives callers precise control
- `BaseException` catches `SystemExit` and `KeyboardInterrupt` — almost never catch it
- `finally` is for cleanup only — never use `return` inside `finally`

🔗 CHECKPOINT QUESTION

You are building a microservice. A `requests.ConnectionError` deep in your HTTP client layer should eventually produce a `503 Service Unavailable` HTTP response to the caller. Trace the exception's journey through three layers (HTTP client → business service → API handler), describing what each layer does with it.

🌟 CHAPTER REVIEW & SUMMARY

What You Learned:

Understanding of the core concepts presented in this chapter including exception handling patterns, their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.
- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **Python Exception Hierarchy — Built-in Exception Tree:** A tree diagram showing Python's built-in exception hierarchy. Root node at top: ...